

Урок 17. Корутины. coroutineScope, supervisorScope, withContext, runBlocking.

07 июля 2020

В этом уроке обсуждаем функции `coroutineScope`, `supervisorScope`, `withContext`, `runBlocking`.

Функции `coroutineScope`, `supervisorScope`, `withContext` очень похожи между собой. Поэтому мы подробно разберем только `coroutineScope`. А для `supervisorScope` и `withContext` проговорим их отличия от `coroutineScope`.

Сразу скажу, что в этом уроке будут в основном теоретические сведения. Практическое использование этих функций мы обсудим, когда будем рассматривать примеры использования корутин в реальном коде.

coroutineScope

Можно выделить два основных назначения `coroutineScope`:

- 1) создает свой scope, что позволяет ограничить распространение ошибки
- 2) позволяет выносить вызов корутин в отдельные (`suspend`) функции

Рассмотрим эти возможности на примере.

В родительской корутине запускается 4 дочерних корутины, и в одной из них (1_2) происходит ошибка.

```
1  scope.launch(CoroutineName("1")) {
2
3      launch(CoroutineName("1_1")) {
4          // ...
5      }
6
7      launch(CoroutineName("1_2")) {
8          // exception
9      }
10
11     launch(CoroutineName("1_3")) {
12         // ...
13     }
14
15     launch(CoroutineName("1_4")) {
16         // ...
17     }
18 }
19 }
```

Ошибка в корутине 1_2 приведет к тому, что отменятся все остальные корутины.

Но что если для нас корутины 1_1 и 1_2 не критичны, и при их ошибках нам не надо отменять все вокруг. Для этого мы можем поместить их в `coroutineScope`:

```
1  scope.launch(CoroutineName("1")) {
2
3      coroutineScope {
4          launch(CoroutineName("1_1")) {
5              // ...
6          }
7
8          launch(CoroutineName("1_2")) {
9              // exception
10         }
11     }
12
13     launch(CoroutineName("1_3")) {
14         // ...
15     }
16
17     launch(CoroutineName("1_4")) {
18         // ...
19     }
20 }
21 }
```

Технически, `coroutineScope` - это `suspend` функция, внутри которой запускается специальная корутин `ScopeCoroutine`. Код, который мы пишем в блоке `coroutineScope`, становится кодом этой корутин и выполняется именно в ней.

Следовательно, в примере выше у нас между корутинами 1 и 1_2 образуется связь:

`Coroutine_1 > ScopeCoroutine > Coroutine_1_2`

а не:

`Coroutine_1 > Coroutine_1_2`

Т.е. `coroutineScope` вклинивает новую корутину `ScopeCoroutine` между нашими родительской и дочерней корутиной, тем самым меняя связь между ними, и, как следствие, поведение при возникновении ошибки.

Основная особенность `ScopeCoroutine` в том, что она не передает ошибку родителю. Т.е. если в корутине `1_2` произойдет ошибка, то она пойдет в `ScopeCoroutine`, которая отменит себя и своих детей (т.е. корутину `1_1`), но своему родителю (корутине `1`) она ошибку передавать не будет. Т.е. ошибка не пойдет дальше вверх по иерархии корутин. Поэтому корутина `1` не отменится сама и не отменит все свои дочерние корутины (`1_3` и `1_4`).

Но! Ошибка из корутины `1_2` не пропадает просто так. `ScopeCoroutine`, хоть и не передаст ее своему родителю, но она передаст ее в свою обертку - `suspend` функцию `coroutineScope`. И `suspend` функция выбросит эту ошибку в наш код в месте своего вызова. И если не поймать ее там, то мы получим стандартное поведение при ошибке в корутине `1`. И в результате все поотменяется.

Чтобы этого не произошло, обернем `coroutineScope` в `try-catch`:

```
1 | scope.launch(CoroutineName("1")) {
2 |
3 |     try {
4 |         coroutineScope {
5 |             launch(CoroutineName("1_1")) {
6 |                 // ...
7 |             }
8 |
9 |             launch(CoroutineName("1_2")) {
10 |                 // exception
11 |             }
12 |         }
13 |     } catch (e: Exception) {
14 |         // ...
15 |     }
16 |
17 |     launch(CoroutineName("1_3")) {
18 |         // ...
19 |     }
20 |
21 |     launch(CoroutineName("1_4")) {
22 |         // ...
23 |     }
24 |
25 | }
```

Таким образом мы смогли остановить распространение ошибки вверх по иерархии корутин.

Если помните, в уроках про обработку ошибок мы пытались сделать то же самое, помещая `launch` в `try-catch`. Но это нам не помогало. Ошибка передавалась под капотом, от дочернего `Job` к родительскому. А `coroutineScope` вытаскивает это из-под капота. Ошибка передается обычным путем, т.е. выбросом исключения в наш код. И мы можем сами решать, ловить это исключение или нет.

Чтобы сделать код лаконичнее, можно `coroutineScope` и все его содержимое вынести в отдельную `suspend` функцию.

```
1 | private suspend fun twoCoroutines() {
2 |     coroutineScope {
3 |         launch(CoroutineName("1_1")) {
4 |             // ...
5 |         }
6 |
7 |         launch(CoroutineName("1_2")) {
8 |             // exception
9 |         }
10 |     }
11 | }
```

Как видите, `coroutineScope` дает нам возможность создавать `suspend` функции, в которых можно запускать корутины без явного указания `scope`. В качестве `scope` будет выступать корутина `ScopeCoroutine`, которая работает внутри `coroutineScope`.

Перепишем пример:

```
1 | scope.launch(CoroutineName("1")) {
2 |
3 |     try {
4 |         twoCoroutines()
5 |     } catch (e: Exception) {
6 |         // ...
7 |     }
8 |
9 |     launch(CoroutineName("1_3")) {
10 |         // ...
11 |     }
12 |
13 |     launch(CoroutineName("1_4")) {
14 |         // ...
15 |     }
16 |
17 | }
```

Отдельно обсудим несколько важных моментов.

`scope`

В примере выше все корутины находятся в рамках одного общего scope, из которого мы вызывали корутину 1. И сохраняется связь от родителя к дочерней. Т.е. вызвав `scope.cancel`, мы каскадно отменим все корутины, в том числе и те, которые находятся внутри `coroutineScope`, т.е. корутины 1_1 и 1_2.

handler

Обработчики `CoroutineExceptionHandler` не работают внутри `coroutineScope`, потому что `ScopeCoroutine` принимает ошибку от корутины 1_2 и говорит, что сможет ее обработать.

Но в случае с `supervisorScope` это поведение меняется, чуть позже обсудим, как именно.

возврат значения

`coroutineScope` может вернуть результат своей работы

```
1 | val result = coroutineScope {
2 |     // ...
3 |     "result value"
4 | }
```

Т.е. внутри `coroutineScope` можно не только новые корутины запускать, но и выполнять обычный код, результат которого станет результатом вызова `coroutineScope`.

suspend функции

Т.к. `coroutineScope` - это `suspend` функция, то ее вызов приостановит поток текущей корутины. Если вам это не надо, то вы можете поместить этот вызов в `launch`:

```
1 | launch {
2 |     coroutineScope {
3 |         // ...
4 |     }
5 | }
```

или в `async`, если ожидаете получить результат:

```
1 | async {
2 |     coroutineScope {
3 |         // ...
4 |         "result value"
5 |     }
6 | }
```

Не забывайте при этом оборачивать `coroutineScope` в `try-catch`, если хотите, чтобы ошибка не распространялась вверх.

Потоки

Код `coroutineScope` выполняется в том же потоке, что и вызвавшая его корутина. А вот продолжить вызвавшую корутину он может в другом потоке (но в том же диспетчере).

supervisorScope

Если `coroutineScope` принимает ошибки от своих дочерних корутин и просто не шлет их дальше в родительскую корутину, то `supervisorScope` даже не принимает ошибку от дочерних. Это приводит к отличиям в обработке ошибок.

Корутина 1_2, которая пытается передать ошибку вверх в `ScopeCoroutine`, получает отрицательный ответ и пытается обработать ошибку сама. Для этого она использует предоставленный ей `CoroutineExceptionHandler`. Если же его нет, то будет крэш, который не поймать никаким `try-catch`.

Поэтому имеет смысл использовать `CoroutineExceptionHandler` внутри `supervisorScope`. В этом случае ошибка попадет в этот обработчик и на этом все закончится. Функция `supervisorScope` не выбросит исключение и не отменит остальные корутины внутри себя (т.е. корутину 1_1 в примере выше).

withContext

`withContext` это `coroutineScope` с возможностью добавить/заменить элементы контекста.

Чаще всего используется для смены диспетчера:

```
1 launch {  
2     // get data from network or database  
3     // ...  
4  
5     withContext(Dispatchers.Main) {  
6         // show data on the screen  
7         // ...  
8     }  
9  
10    // ...  
11 }
```

В `launch` выполняется какой-либо тяжелый код для получения данных. Далее с помощью `withContext` переключаемся на `Main` поток и отображаем данные на экране. После чего корутина продолжает выполняться в своем диспетчере.

Либо корутина может выполняться в `Main` потоке, а с помощью `withContext` мы можем переключиться на `IO` поток, чтобы сделать запрос на сервер или в БД, и получить результат обратно в `Main` поток.

Когда будем разбирать реальные примеры кода, увидим, как используются эти механизмы.

runBlocking

`runBlocking` запускает корутину, которая блокирует текущий поток, пока не завершит свою работу (и пока не дожидается завершения дочерних корутин).

В `Android` разработке такая корутина нужна для написания `unit` тестов, чтобы удерживать поток, в котором выполняется тест. Иначе тест просто не дожидается завершения выполнения вызываемых корутин и завершится. Подробнее об этом мы еще поговорим в уроке про тестирование.

Если в `runBlocking` произойдет ошибка, то все его дочерние корутины поотменяются, а сам он выбросит исключение.

Также он может вернуть результат своей работы.

`runBlocking` не предполагается к использованию в основном коде `Android` приложения.